

CSI3105 - Software Testing

Case Study

Title: Event Booking Software
Component System

Samarakoon Mudiyanseelage Sanuri Dinusha

Table of Contents

Introduction.....	2
Testing Approach.....	2
Questions 1.....	3
Test Cases for EQ and BVA	4
Question 2	6
Control Flow Graphs (CFGs)	6
Structural Testing	11
Question 3	15
Test case execution	15
Test plan and defect report.....	15
Results	16
Found Errors.....	17
Conclusion	17

Introduction

This assignment is focused on testing a command-line-based program with multiple classes. The class with the main method is “Planner” class, which executes the program and requests the user input to proceed with scheduling a meeting or a vacation, check room or employee availability, or check the agendas of specific rooms or employees.

Mainly, the equivalent classes and boundary value analysis is applied to identify the test cases and implemented them as programming mistakes are often occurred at boundaries. Furthermore, this report only focuses only testing values from the equivalent classes and their boundaries to ensure that the processing of values is done as intended. The deliverables of this assignment are the report containing test cases, statement and branch coverages of 2 classes, Control Flow Graphs of classes, defect report, and test results.

Testing Approach

Most of the test cases that are mentioned in this report were tested in Eclipse software using JUnit and Mockito. The programming language used to implement the test cases is Java language.

Questions 1

What are you doing to address back-box testing ect?

The black-box testing aspect is covered through the equivalent (EQ) classes and boundary value analysis (BVA). The EQ classes only considers weak robust testing, so that both valid and invalid classes are considered and only one value from each class is considered as well. For BVA, all the possible boundaries near and at the boundary values of upper and lower bounds are considered. Since, the code only displays the error messages for the first invalid user input of each time, several test cases required to be tabulated below to verify the boundaries and the value processing.

Equivalence (EQ) Classes

Valid Equivalence Classes:

M1 = {month : 1 <= month <= 12}

L1 = {(year % 4 == 0 && year % 100 != 0 || year % 400 == 0) && month == 2 : 1 <= day <= 29}

NL1 = {!(year % 4 == 0 && year % 100 != 0 || year % 400 == 0) && month == 2 : 1 <= day <= 28}

MT1 = {month ∈ {4, 6, 9, 11} : 1 <= day <= 30}

MO1 = {month ∈ {1, 3, 5, 7, 8, 10, 12} : 1 <= day <= 31}

T1 = {booking_time : 0 <= booking_time <= 23}

Invalid Equivalence Classes:

M2 = {month : month < 1}

M3 = {month : month > 12}

L2 = {(year % 4 == 0 && year % 100 != 0 || year % 400 == 0) && month == 2 : day > 29}

NL2 = {!(year % 4 == 0 && year % 100 != 0 || year % 400 == 0) && month == 2 : day > 28}

D1 = {day : day < 1}

MT2 = {month ∈ {4, 6, 9, 11} : day > 30}

MO2 = {month ∈ {1, 3, 5, 7, 8, 10, 12} : day > 31}

T2 = {booking_time : booking_time < 0}

T3 = {booking_time : booking_time > 23}

Boundary Value Analysis (BVA)

EQ Class(es)	Invalid Near Lower Bound Value	Lower Bound Value	Valid Near Lower Bound Value	Valid Near Upper Bound Value	Upper Bound Value	Invalid Near Upper Bound Value
M1, M2, M3	0	1	2	11	12	13
L1, L2, D1	0	1	2	28	29	30
NL1, NL2, D1	0	1	2	27	28	29
MT1, MT2, D1	0	1	2	29	30	31
MO1, MO2, D1	0	1	2	30	31	32
T1, T2, T3	-1	0	1	22	23	24

Table 1: BVA for EQ classes

Test Cases for EQ and BVA

Test Cases for scheduleMeeting() Function

ID	Option	Month	Day	Start Time	End Time	Room	Attendee	Description	Expected Output	Passed/Failed
EQ1	1	5	10	13	14	ML5.123	Edith Cowan Mark Colin done	PhD	"Enter a description for the meeting:"	Passed
EQ2	1	-2	-3	22	27	ML5.123	Edith Cowan done	PhD	"Day does not exist."	Passed
EQ3	1	15	33	-5	-6	ML5.123	Edith Cowan done	PhD	"Day does not exist."	Passed

Table 2: EQ test cases and results for scheduleMeeting() function

Test Cases for scheduleVacation() Function

ID	Option	sMonth	sDay	eMonth	eDay	Absentee	Expected Output	Passed/Failed
BVA1	2	1	0	1	2	Jaci Johnston	"Day does not exist."	Passed
BVA2	2	0	1	0	4	Jaci Johnston	"Month does not exist."	Passed
BVA3	2	2	1	2	2	Jaci Johnston	"Enter a person's	Passed

							name, or cancel to cancel the request:”	
--	--	--	--	--	--	--	--	--

Table 3: BVA test cases and results for scheduleVacation() function

Test Cases for checkRoomAvailability() Function

ID	Option	Month	Day	Start Time	End Time	Expected Output	Passed/ Failed
BVA4	3	2	28	-1	1	“Illegal hour.”	Passed
BVA5	3	2	29	0	2	“The rooms available at the specified time are: ”	Passed
BVA6	3	2	28	0	-1	“Illegal hour.”	Passed

Table 4: BVA test cases and results for checkRoomAvailability() function

Test Cases for checkEmployeeAvailability() Function

ID	Option	Month	Day	Start Time	End Time	Expected Output	Passed/ Failed
BVA7	4	2	30	1	2	“Day does not exist.”	Failed
BVA8	4	2	1	22	23	“Ashley Martin”	Passed
BVA9	4	2	2	22	24	“Illegal hour.”	Passed
BVA10	4	2	2	23	0	“Ashley Martin”	Failed

Table 5: BVA test cases and results for checkEmployeeAvailability() function

Test Cases for checkAgendaRoom() Function

ID	Option	Month	Day	Room	Expected Output	Passed/ Failed
BVA11	5	11	29	ML5.123	“No Meetings booked on this date.”	Passed
BVA12	5	11	30	ML5.123	“No Meetings booked on this date.”	Failed
BVA13	5	13		all ML5.123	“Month does not exist.”	Failed

Table 6: BVA test cases and results for checkAgendaRoom() function

Test Cases for checkAgendaPerson() Function

ID	Option	Month	Day	Person	Expected Output	Passed/ Failed
BVA14	6	11	31	Mark Colin	"Day does not exist."	Failed
BVA15	6	12	30	Mark Colin	"No Meetings booked on this date."	Passed
BVA16	6	12	31	Mark Colin	"No Meetings booked on this date."	Passed
BVA17	6	12	32	Mark Colin	"Day does not exist."	Failed

Table 7: BVA test cases and results for checkAgendaPerson() function

As most of the boundary values are incorrectly defined, the actual outputs were different than the expected outputs. It was clearly seen for hours, days, and months. Additionally, leap years are not considered, so, February cannot have 29 as the last day as well. Even though November has 30 days, it is also marked as an invalid day. Since programmers tend to make mistakes at boundary values, defining clear and precise boundaries must be verified through BVA techniques.

Question 2

Control Flow Graphs (CFGs)

CFGs for every class except the "Planner" class are included here.

CFG for "Meeting" class

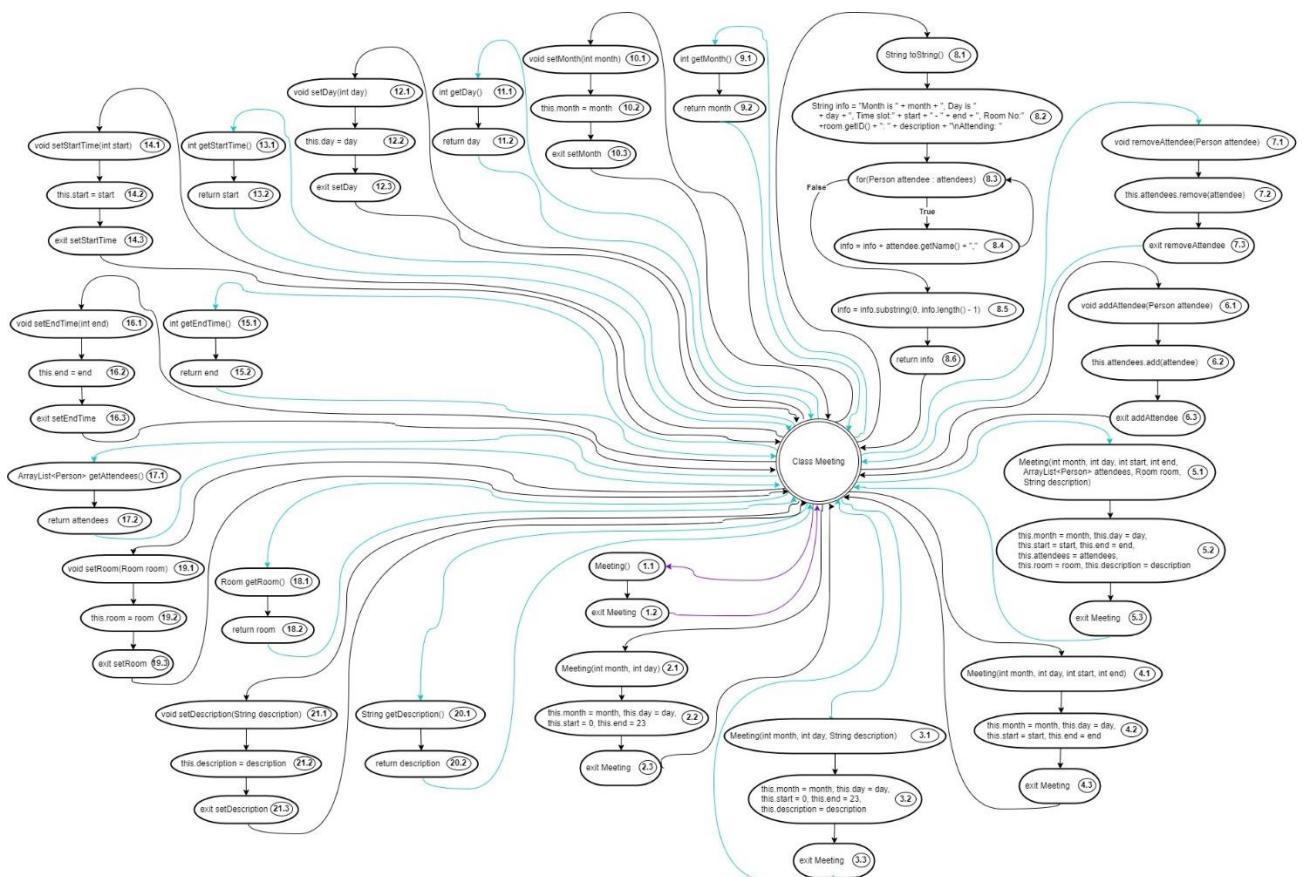


Figure 1: CFG for "Meeting" class

CFG for “Calendar” class

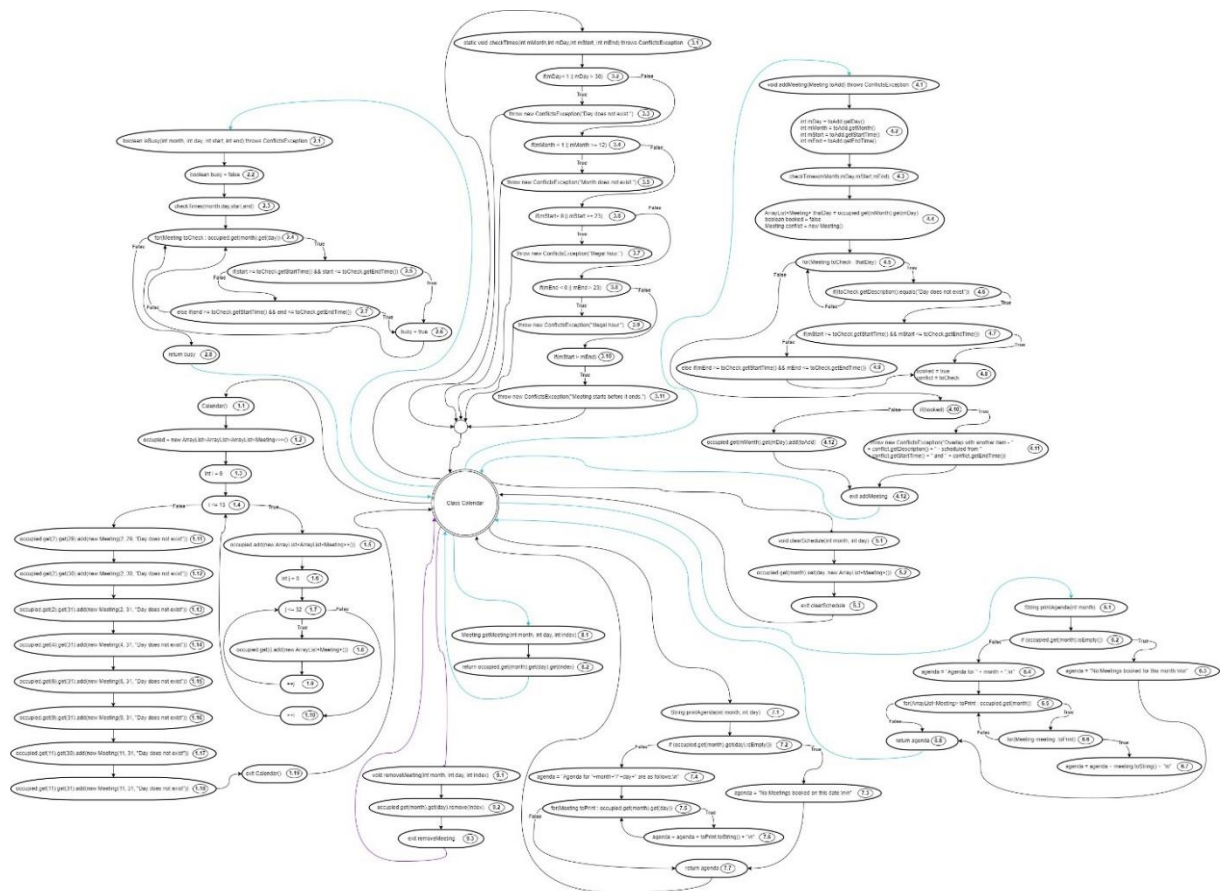


Figure 2: CFG for “Calendar” class

CFG for "ConflictsException" class

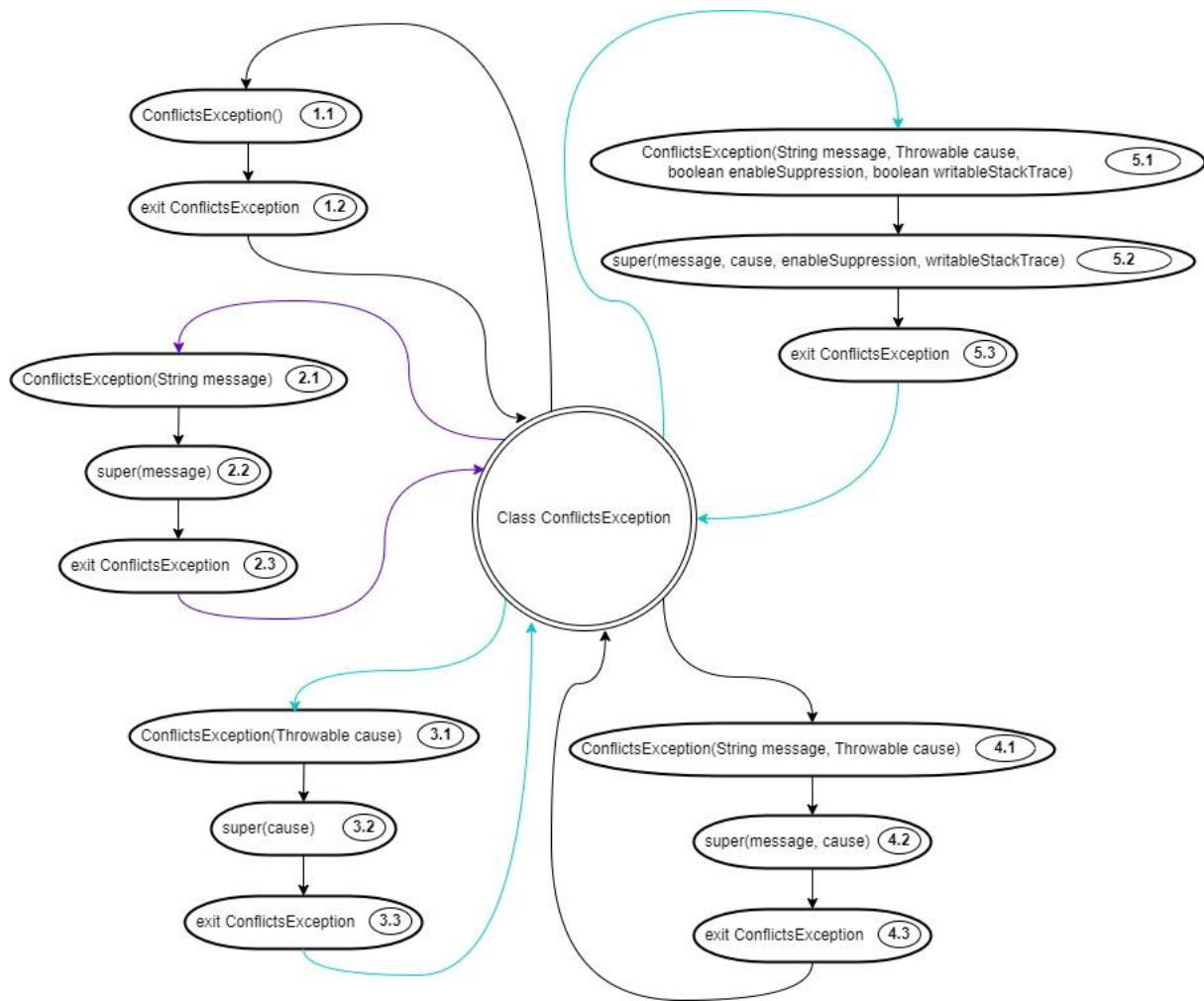


Figure 3: CFG for "ConflictsException" class

CFG for "Room" class

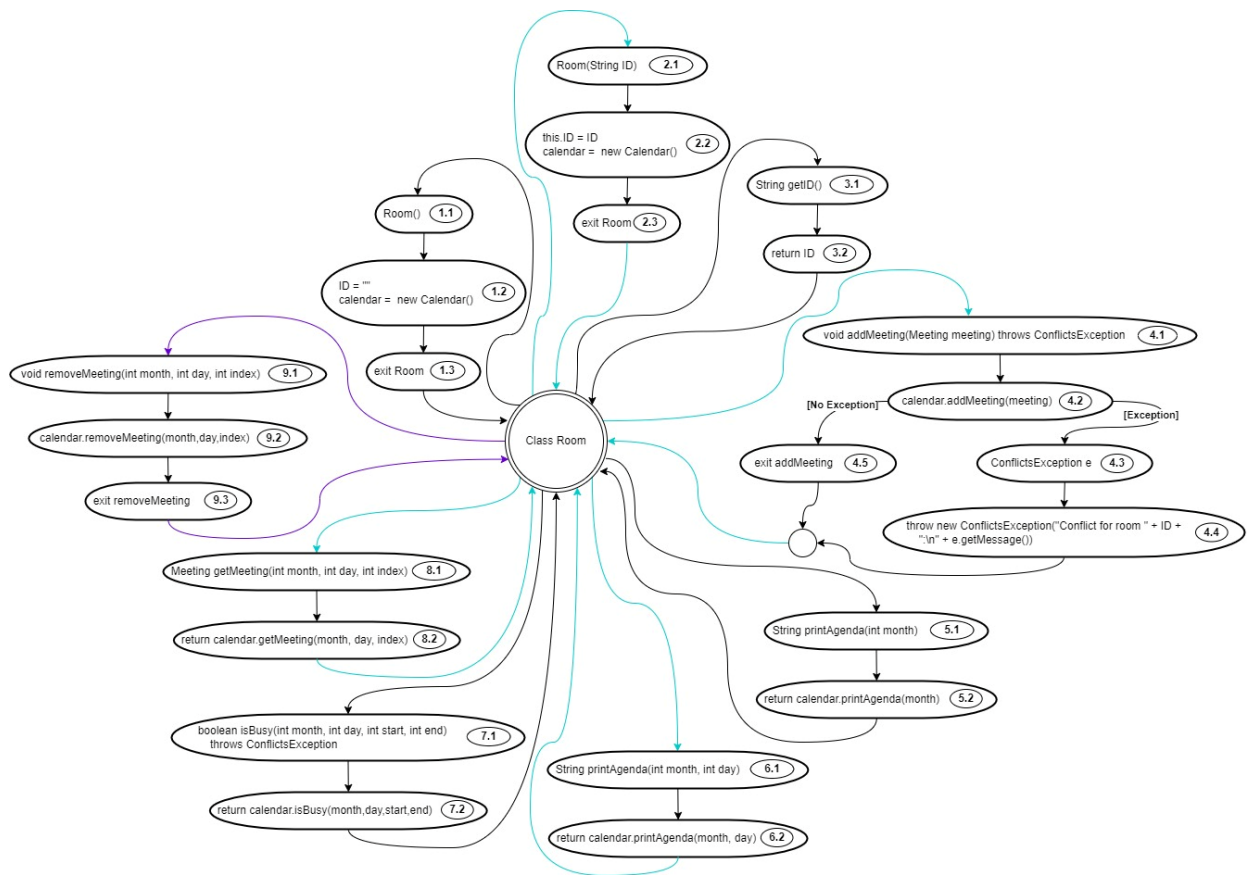


Figure 4: CFG for "Room" class

CFG for "Person" class

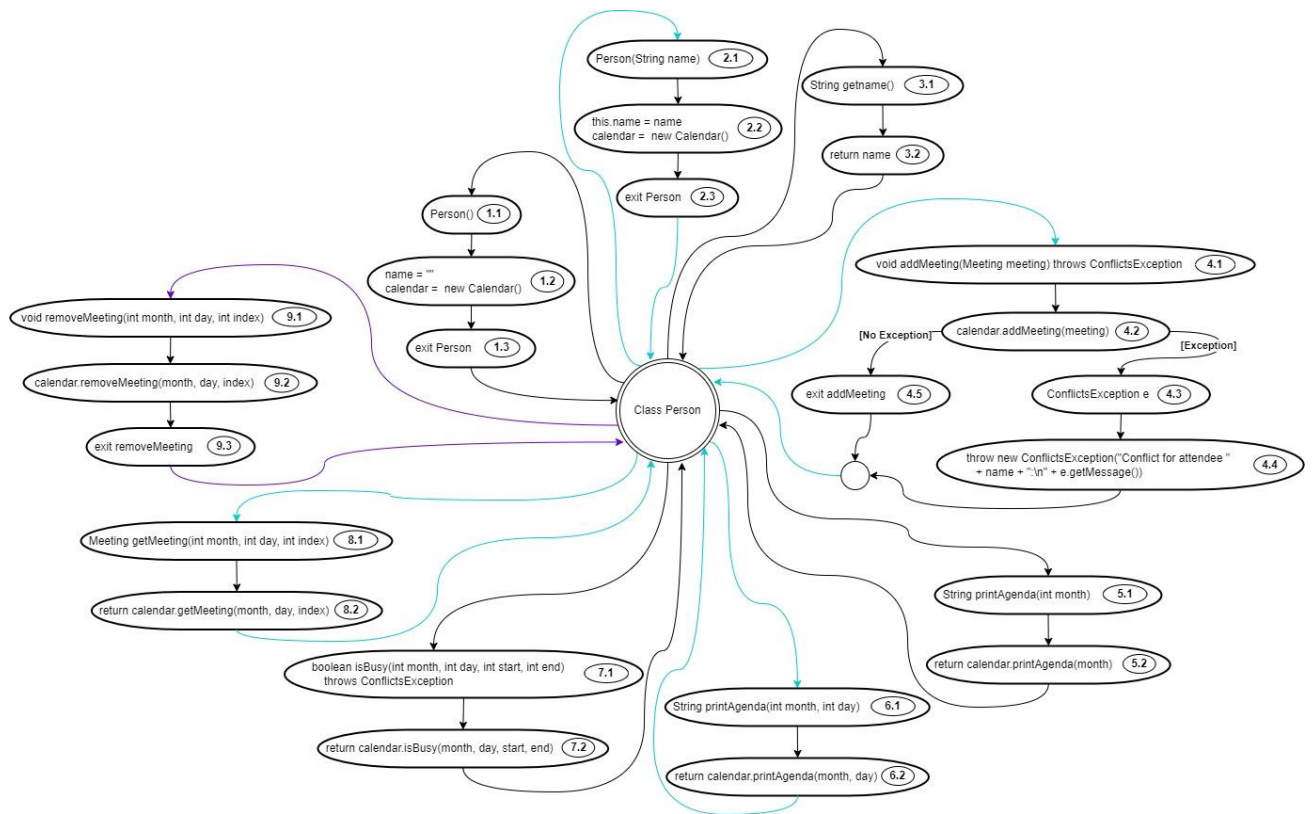


Figure 5: CFG for "Person" class

CFG for "Organization" class

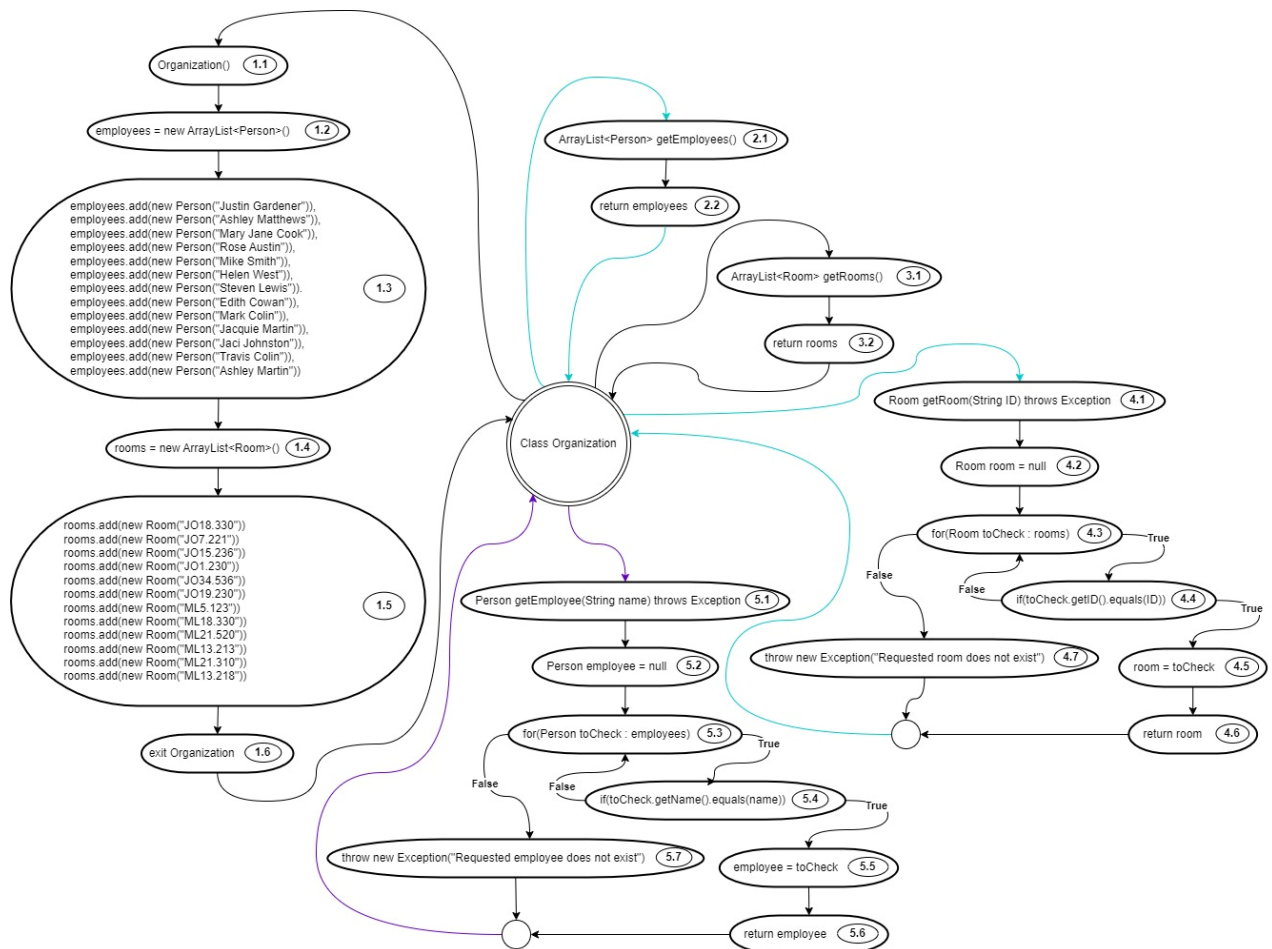


Figure 6: CFG for "Organization" class

Structural Testing

The structural testing aspect was covered only for 2 classes as the assignment has stated. Therefore, statement coverage and branch coverage of 2 classes were included with the paths to execute. Statement coverage considers all the assignment statements and conditional statements. Although every condition is covered, but both true and false branches are considered in branch coverage. So, branch coverage includes all branches of every condition as well. Additionally, the inputs and expected outputs are stated to understand the output of the path.

Chosen 2 classes for unit tests that covers structural testing:

1. "Organization" class
2. "Meeting" class

Statement Coverage for “Organization” Class

Input	Path	Expected Output
-	1.1 – 1.2 – 1.3 – 1.4 – 1.5 – 1.6 – 2.1 – 2.2	“Justin Gardener Ashley Matthews Mary Jane Cook Rose Austin Mike Smith Helen West Steven Lewis Edith Cowan Mark Colin Jacquie Martin Jaci Johnston Travis Colin Ashley Martin”
-	1.1 – 1.2 – 1.3 – 1.4 – 1.5 – 1.6 – 3.1 – 3.2	"JO18.330 JO7.221 JO15.236 JO1.230 JO34.536 JO19.230 ML5.123 ML18.330 ML21.520 ML13.213 ML21.310 ML13.218"
“ML5.123”	1.1 – 1.2 – 1.3 – 1.4 – 1.5 – 1.6 – 4.1 – 4.2 – 4.3 – 4.4 – 4.5 – 4.6	“ML5.123”
“Mark Colin”	1.1 – 1.2 – 1.3 – 1.4 – 1.5 – 1.6 – 5.1 – 5.2 – 5.3 – 5.4 – 5.5 – 5.6	“Mark Colin”

Table 8: Statement coverage paths and expected output for “Organization” class

Statement Coverage for “Meeting” Class

Input	Path	Expected Output
month = 2, day = 10, start = 11, end = 12, room = “ML5.123”	1.1 – 1.2 – 10.1 – 10.2 – 10.3 – 12.1 – 12.2 – 12.3 – 14.1 – 14.2 – 14.3 – 16.1 – 16.2 – 16.3 – 19.1 – 19.2 – 19.3 – 9.1 – 9.2 – 11.1 – 11.2 – 13.1 – 13.2 – 15.1 – 15.2 – 18.1 – 18.2	2 10 11 12 “ML5.123”
month = 2, day = 10, description = “description”, room = “ML5.123”	2.1 – 2.2 – 2.3 – 21.1 – 21.2 – 21.3 – 19.1 – 19.2 – 19.3 – 8.1 – 8.2 – 8.3 – 8.5 – 8.6	"Month is 2, Day is 2, Time slot: 0 – 23, Room No:ML5.123: description

		Attending:"
month = 2, day = 10, description = "description"	3.1 – 3.2 – 3.3 – 20.1 – 20.2	"description"
attendee = "Mark Colin", month = 2, day = 10, start = 11, end = 12, attendees = [attendee], room = "ML5.123", description = "description"	6.1 – 6.2 – 6.3 – 5.1 – 5.2 – 5.3 – 7.1 – 7.2 – 7.3 – 8.1 – 8.2 – 8.3 – 8.5 – 8.6	"Month is 2, Day is 2, Time slot: 11 – 12, Room No:ML5.123: description Attending:"
month = 2, day = 10, start = 11, end = 12,	4.1 – 4.2 – 4.3 – 17.1 – 17.2	[]

Table 9: Statement coverage paths and expected output for "Meeting" class

Branch Coverage for "Organization" Class

Input	Path	Expected Output
-	1.1 – 1.2 – 1.3 – 1.4 – 1.5 – 1.6 – 2.1 – 2.2	"Justin Gardener Ashley Matthews Mary Jane Cook Rose Austin Mike Smith Helen West Steven Lewis Edith Cowan Mark Colin Jacquie Martin Jaci Johnston Travis Colin Ashley Martin"
-	1.1 – 1.2 – 1.3 – 1.4 – 1.5 – 1.6 – 3.1 – 3.2	"JO18.330 JO7.221 JO15.236 JO1.230 JO34.536 JO19.230 ML5.123 ML18.330 ML21.520 ML13.213 ML21.310 ML13.218"
"JO18.330"	1.1 – 1.2 – 1.3 – 1.4 – 1.5 – 1.6 – 4.1 – 4.2 – 4.3 – 4.4 – 4.5 – 4.6	"JO18.330"
"Justin Gardener"	1.1 – 1.2 – 1.3 – 1.4 – 1.5 – 1.6 – 5.1 – 5.2 – 5.3 – 5.4 – 5.5 – 5.6	"Justin Gardener"
rooms = ["JO18.330"], ID = "ML5.12345"	4.1 – 4.2 – 4.3 – 4.4 – 4.3 – 4.7	"Requested room does not exist"

employees = ["Justin Gardener"], name = "Mark Colin"	5.1 – 5.2 – 5.3 – 5.4 – 5.3 – 5.7	"Requested employee does not exist"
---	-----------------------------------	-------------------------------------

Table 10: Branch coverage paths and expected output for "Organization" class

Branch Coverage for "Meeting" Class

Input	Path	Expected Output
month = 2, day = 10, start = 11, end = 12, room = "ML5.123"	1.1 – 1.2 – 10.1 – 10.2 – 10.3 – 12.1 – 12.2 – 12.3 – 14.1 – 14.2 – 14.3 – 16.1 – 16.2 – 16.3 – 19.1 – 19.2 – 19.3 – 9.1 – 9.2 – 11.1 – 11.2 – 13.1 – 13.2 – 15.1 – 15.2 – 18.1 – 18.2	2 10 11 12 "ML5.123"
month = 2, day = 10, description = "description", room = "ML5.123"	2.1 – 2.2 – 2.3 – 21.1 – 21.2 – 21.3 – 19.1 – 19.2 – 19.3 – 8.1 – 8.2 – 8.3 – 8.5 – 8.6	"Month is 2, Day is 2, Time slot: 0 – 23, Room No:ML5.123: description Attending:"
month = 2, day = 10, description = "description"	3.1 – 3.2 – 3.3 – 20.1 – 20.2	"description"
attendee = "Mark Colin", month = 2, day = 10, start = 11, end = 12, attendees = [attendee], room = "ML5.123", description = "description"	6.1 – 6.2 – 6.3 – 5.1 – 5.2 – 5.3 – 7.1 – 7.2 – 7.3 – 8.1 – 8.2 – 8.3 – 8.5 – 8.6	"Month is 2, Day is 2, Time slot: 11 – 12, Room No:ML5.123: description Attending:"
month = 2, day = 10, start = 11, end = 12,	4.1 – 4.2 – 4.3 – 17.1 – 17.2	[]
attendee = "Mark Colin", month = 2, day = 10, start = 11, end = 12, attendees = [attendee], room = "ML5.123", description = "description"	6.1 – 6.2 – 6.3 – 5.1 – 5.2 – 5.3 – 8.1 – 8.2 – 8.3 – 8.4 – 8.3 – 8.5 – 8.6	"Month is 2, Day is 2, Time slot: 11 – 12, Room No:ML5.123: description Attending: Mark Colin "

Table 11: Branch coverage paths and expected output for "Meeting" class

Commencing structural testing on such classes is not easy as they contain multiple methods. However, some paths were assumed that pre-configured variables such as an ArrayList is existing to elaborate the execution of the paths. Otherwise, lengthy repeating steps would have been included in the paths as well. Therefore, assumptions eased the structural testing.

Question 3

The exceptions also have been addressed in the structural testing. Although some unknown exceptions were thrown due to the errors of incorrectly defined boundaries in the program.

Test case execution

The tests must be executed like this. The written test cases must be implemented under a “tests” folder and conducted in Eclipse using the same inputs to administer the outputs. Then, the actual results should be recorded in a table and identify the locations of the code where the errors exists and where the expected and actual outputs differ. After that, modify and amend the program and test accordingly until the desired outputs are delivered. Moreover, the relevant CFG must be referred for the structural testing, as the testing paths are based on the statement block number that is mentioned on the CFG. Likewise, the functionality and the structure of the program can be tested and rectify the code as required.

Test plan and defect report

Defect Report

What defects or abnormalities were found? (put in a table)

Significant defects were found in the code where boundary values have been involved, therefore, programming errors must have been caused. This has led to consider valid dates and times as invalid which makes it impossible to use them to be included in a plan. Moreover, due to these errors, index errors and exceptions are thrown and interrupts the programme. Therefore, such errors must be corrected for a smooth flow and correct outputs.

Number	Class	Defects/ Abnormalities	Correction(s)
1	Calendar	Have 13 months and 32 days per month	Remove the equal sign to correct as $i < 13$ and $j < 32$
2	Calendar	Had not considered leap years so February 29 th is invalid.	Consider leap years
3	Calendar	November 30 th is marked as an invalid day. The code tried to mark November 31 st as invalid for twice.	A valid day
4	Calendar	Any day including 30 th is taken as illegal.	$mDay > 31$
5	Calendar	23 must be a valid hour, it is considered as an illegal hour for a start time.	$mStart > 23$
6	Planner	Case sensitive for alphabetical user inputs.	User input must be validated and cleansed to ignore case sensitivity.
7	Planner	Only the first invalid input throws any error messages.	Must consider all inputs and display output only if they are validated.

Results

There are 20 test cases that have been tabulated, implemented, and executed to verify whether they pass or fail based on the expected outcome vs actual outcome. The tabulated results of EQ and BVA test cases can be found in Table 2 – 7 and Figure 7 and 8 indicates the success rate of them after being executed using Junit and Mockito in Eclipse. The 6 failed BVA test cases are related to hour, date, and month values. 0-23 hours; 1-31 days including 30th November and 29th February for leap years; and 1-12 months are the correct boundaries to be implemented.

EQ Tests from scheduleMeeting()

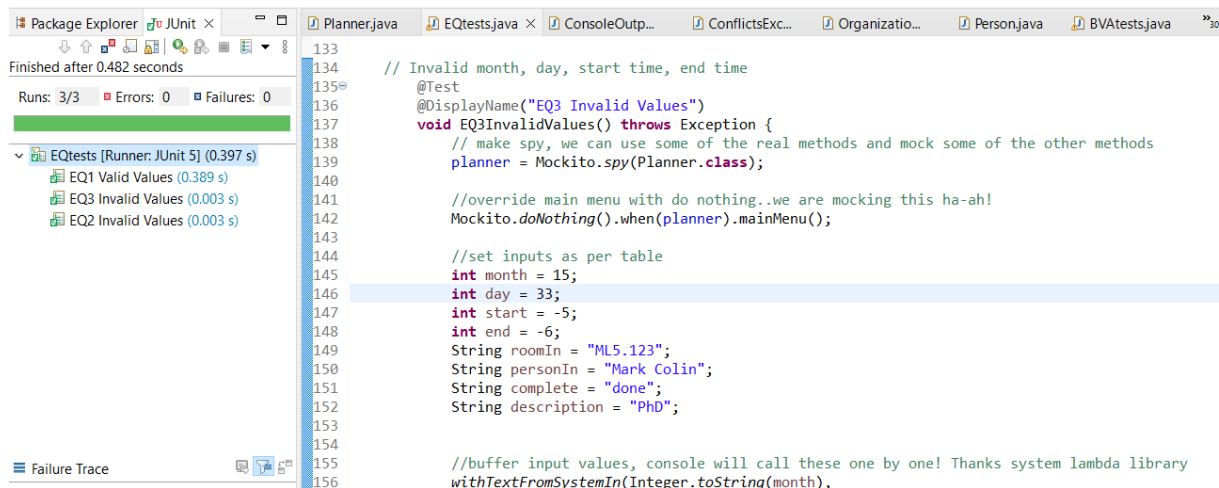


Figure 7: EQ tests run using the scheduleMeeting() function

EQ Tests from scheduleVacation(), checkRoomAvailability(), checkEmployeeAvailability(), checkAgendaRoom(), and checkAgendaPerson()

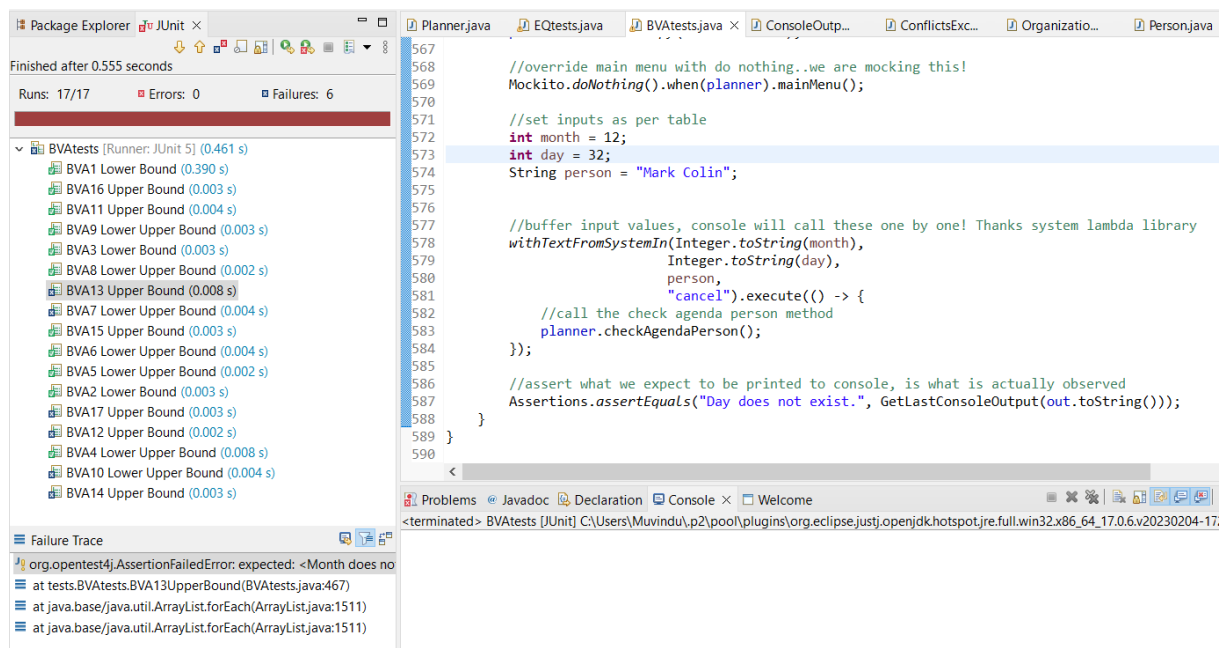


Figure 8: BVA tests run using the scheduleMeeting(), checkRoomAvailability(), checkEmployeeAvailability(), checkAgendaRoom(), and checkAgendaPerson() functions

Found Errors

- Exceeding the maximum index value.
- Incorrectly defined boundaries (i.e. boundary implementations with condition statements).
- Unable to get a nullified ID.

Conclusion

The testing process began with writing test cases and identifying test paths. Then, with weak robust equivalence class partitioning and boundary value analysis (BVA) were applied to derive the test inputs. Likewise, all the aspects of partitions and boundaries were covered by 20 test cases that were tabulated with expected outcomes and implemented them in Eclipse using Junit and Mockito to discover that, programming errors are present around boundary values. Therefore, differences of the actual and expected outputs could be found in 6 out of 17 BVA-based test cases. The test paths of the chosen 2 classes “Organization” and “Meeting” includes the exception handling as well. At the end, learning to automate test cases through Mockito library saved more time while testing 20 test cases.